

# A Simulation Framework for the Multi-Period Vehicle Routing Problem with Profits in Smart Waste Collection

Afonso Cruz Fernandes<sup>1</sup>(✉), Héctor Bonilla-Londoño<sup>2</sup>, Diana Jorge<sup>2</sup>, Manuel Lopes<sup>1</sup>, Pedro A. Santos<sup>1</sup>, and Tânia Ramos<sup>2</sup>

<sup>1</sup> INESC-ID, Instituto Superior Técnico, Universidade de Lisboa, Lisboa, Portugal  
{afonso.fernandes,manuel.lopes,pedro.santos}@tecnico.ulisboa.pt

<sup>2</sup> CEGIST, Instituto Superior Técnico, Universidade de Lisboa, Lisboa, Portugal  
hector.fabio.bonilla.londono@tecnico.ulisboa.pt  
{diana.jorge,tania.p.ramos}@tecnico.ulisboa.pt

**Abstract.** Smart waste collection is a stochastic multi-period routing problem: bins fill between decisions, sensing is imperfect, and collecting a bin today changes the opportunities available tomorrow. We introduce WSmartRoute+, a Python simulation framework for evaluating policies for this setting. Each policy combines interchangeable mandatory-bin selection, profitable route construction, and route-improvement stages. This structure allows one decision to vary while the demand realization and other stages remain fixed. The reported benchmark crosses five selection variants, eight classical constructors, two improvers, two waste-generation processes, and three municipal network sizes at a 30-day horizon. The results show a substantially larger observed efficiency spread across selection variants than across constructors, exposing a clear trade-off between collection efficiency and overflow risk. Matched-demand improver configurations differ substantially, although stochastic upstream construction prevents causal attribution to that stage. A selected subset is followed for 90 days. We report matched within-policy comparisons only because selection on 30-day performance precludes an unbiased cross-constructor ranking. Learned constructors share the framework interface but are not bench-marked here.

## 1 Introduction

Waste collection routing couples transport cost to service reliability. Bins accumulate material at different rates, sensor readings are imperfect, vehicle capacity is limited, and a collection changes the following day’s state. A useful evaluation must therefore follow a policy over time instead of scoring an isolated route.

Existing work covers exact models, meta-heuristics, sensor placement, and data-driven planning for waste collection [6,7,15,17]. Neural combinatorial optimization (NCO) adds learned constructive and improvement methods for routing [24]. Results remain difficult to compare when studies use different operational dynamics, demand samples, or tightly coupled selection and routing rules.

WSmartRoute+ provides a simulation layer for the stochastic MPVRPP. It models daily accumulation, noisy sensing, route execution, overflow and loss, then carries the resulting state into the next decision. Policy components use registries and a common interface, allowing classical or learned constructors to be combined with the same selection and improvement stages.

We evaluate the classical baseline on bin locations, road distances, and historical fill records from two Portuguese municipalities. The complete 30-day factorial design contains 480 runs. A performance-selected subset is also observed for 90 days, with the corresponding inferential limits stated explicitly.

This paper makes the following contributions:

- A reproducible simulator for stochastic multi-period waste collection, including bin-level accumulation, sensing error, overflow, loss, and state transitions after collection.
- A three-stage policy interface that separates mandatory selection, profitable route construction, and route improvement, with common demand realizations for matched comparisons.
- A classical baseline spanning five selection variants, eight constructors, two improvers, two demand processes, and three municipal network sizes; learned constructors can use the same interface in future comparisons.
- An integrity-aware analysis that excludes failed runs by whole scenario cells, balances every marginal comparison, and limits the selected 90-day follow-up to claims supported by its design.

## 2 Problem Definition

We model the Multi-Period Vehicle Routing Problem with Profits (MPVRPP) as a constrained stochastic optimization problem over  $D$  days.

### 2.1 Problem State and Dynamics

Let  $\mathcal{G} = (\mathcal{V}, \mathcal{E})$  be a complete graph with depot  $v_0$  and bins  $\mathcal{V} \setminus \{v_0\}$ . Bin  $i$  has location  $\vec{l}_i \in \mathbb{R}^2$ , capacity  $C_i$ , and waste level  $w_{i,d} \in [0, C_i]$  on day  $d$ . The system state is  $\vec{w}_d = [w_{1,d}, \dots, w_{V,d}]^\top$ .

Daily accumulation  $\delta_{i,d} \sim P_i$  is bin-specific and stochastic. If bin  $i$  is collected on day  $d$ , the next day's level contains only the next increment; otherwise accumulation continues to capacity:

$$w_{i,d+1} = \begin{cases} \delta_{i,d+1} & \text{if } v_i \in \mathcal{A}_d \\ \min(w_{i,d} + \delta_{i,d+1}, C_i) & \text{if } v_i \notin \mathcal{A}_d \end{cases} \quad (1)$$

### 2.2 Optimization Objective

On day  $d$ , the policy produces up to  $K$  routes. Route  $k$  visits  $T_{d,k}$  bins and starts and ends at  $v_0$ :

$$\mathcal{A}_{d,k} = (a_{0,k}, a_{1,k}, \dots, a_{T_{d,k},k}, a_{T_{d,k}+1,k}), \quad a_{0,k} = a_{T_{d,k}+1,k} = v_0. \quad (2)$$

Let  $\mathcal{A}_{d,\cdot}$  denote the union of non-depot visits. The daily problem is

$$\begin{aligned} \text{maximize} \quad & \mathcal{P}(\mathcal{A}_d, \mathbf{w}_d) = r_w \sum_{v_i \in \mathcal{A}_{d,\cdot}} w_{i,d} - c_{km} \sum_{k=1}^K \sum_{t=0}^{T_{d,k}} \text{dist}(a_{t,k}, a_{t+1,k}) \\ \text{subject to} \quad & a_{t,k} \neq a_{t',k'} \quad \forall (t,k) \neq (t',k'), \quad 1 \leq t \leq T_{d,k}, \quad 1 \leq t' \leq T_{d,k'}, \quad (3) \\ & \sum_{t=1}^{T_{d,k}} w_{a_{t,k},d} \leq Q_k \quad \forall k \in \{1, \dots, K\}. \end{aligned}$$

Here  $r_w$  is revenue per kilogram,  $c_{km}$  is travel cost per kilometer, and  $Q_k$  is vehicle  $k$ 's capacity. Optional service distinguishes the VRPP from the classical VRP: the policy chooses a profitable subset as well as its routes [1].

Because each collection changes the next state, policy  $\pi$  maximizes expected cumulative profit over the horizon:

$$\max_{\pi} \mathbb{E} \left[ \sum_{d=1}^D \mathcal{P}_d(\mathcal{A}_{d,\cdot}, \mathbf{w}_d) \right], \quad (4)$$

where  $\mathcal{P}_d$  is defined in (3).

### 3 Related Work

#### 3.1 Neural Combinatorial Optimization

Learned solvers fit the framework at either route construction or improvement; the present experiment does not benchmark them. Constructive solvers may decode a route sequentially, as in Pointer Networks, the Attention Model, and POMO [23,11,12], or predict edge scores or a complete solution representation for a separate decoder [9,10,21]. Learning-to-search methods instead refine an existing solution through learned local moves [16]. These families use supervised, reinforcement, or unsupervised objectives and differ materially in inference cost and generalization beyond their training sizes [8,24]. The shared constructor interface permits a later controlled comparison with the classical baseline reported here.

#### 3.2 Multi-Period Routing with Profits

Classical VRP formulations require service at every customer, whereas profitable routing jointly chooses whom to serve and how to route the chosen subset [1]. Prior multi-period profitable routing distributes service across periods under inter-period scheduling constraints [25]. Waste collection introduces a different coupling: service resets a bin's state, while deferral raises its future collection value and overflow risk. The policy must trade immediate routing cost against the future system state. Waste-routing studies have developed exact and heuristic methods

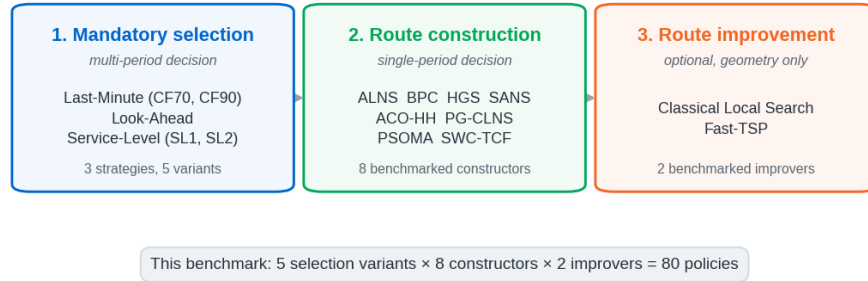
for realistic operational constraints [6,7,15,17], but reproducible comparisons across selection, construction, and improvement rules remain difficult when each study embeds those decisions in a monolithic solver. WSmartRoute+ addresses that evaluation gap by separating the decisions and replaying identical stochastic demand across composed policies.

## 4 Methodology

A WSmartRoute+ policy composes three independent stages, each drawn from its own plugin registry (Fig. 1). The first is *mandatory selection*: given the sensed fill level of every bin, decide which bins *must* be serviced today. The second is *route construction*: build a route over those mandatory bins, free to add any further bin whose waste repays the detour needed to reach it. The third, optional, stage is *route improvement*: take the constructed route and refine it.

The stages answer different questions. Selection is multi-period because collecting today forgoes a larger, cheaper-per-kilogram load tomorrow. Construction solves the current profitable-routing problem, and improvement refines its geometry. Their composition lets an experiment vary one stage while holding the others fixed. Registered implementations therefore define a policy *space* instead of one fixed solver.

This work benchmarks a deliberately small, representative subset of that space: three selection strategies (in five threshold variants), eight route constructors spanning exact methods, meta-heuristics and hyper-heuristics, and two route improvers.



The framework registries expose a larger implementation space; this figure shows only the fully crossed benchmark subset.

**Fig. 1.** The policy configuration space. A policy is one choice from each of the three stages; the experiments in Sect. 5 cross the highlighted subset exhaustively.

#### 4.1 Mandatory Selection Strategies

All three strategies decide from *sensed* fill levels, never from the true ones, and all three must estimate whatever they need about accumulation from the history the simulation has produced so far. Neither the generating distribution nor its parameters are ever revealed to them. The three were chosen to span the conceptual space rather than to be exhaustive: one reactive, one statistical, one simulation-based.

*Last-Minute (LM)*. The simplest possible rule, first proposed in [17], and the reactive baseline: a bin becomes mandatory when its sensed fill level crosses a fixed critical-fill threshold, judged in complete isolation from every other bin and from any estimate of how fast it is filling. Formally, bin  $i$  is mandatory on day  $d$  iff  $\hat{w}_{i,d} \geq \tau$ , for sensed fill  $\hat{w}_{i,d}$  and threshold  $\tau$ . We test  $\tau = 0.7$  (CF70) and  $\tau = 0.9$  (CF90). The threshold is the only knob, and it trades overflow risk against visit frequency directly, with no model connecting the two.

*Service-Level (SL)*. A statistical rule that projects forward rather than reacting. For each bin it maintains online estimates of the mean daily accumulation  $\hat{\mu}_i$  and its standard deviation  $\hat{\sigma}_i$  from the days observed so far, and flags bin  $i$  as mandatory when a conservative projection of its fill  $n_d$  days ahead already exceeds capacity:

$$\hat{w}_{i,d} + n_d \hat{\mu}_i + z \hat{\sigma}_i \sqrt{n_d} \geq 100\%, \quad (5)$$

where  $z$  is a fixed safety coefficient. We test projection horizons  $n_d = 1$  (SL1) and  $n_d = 2$  (SL2), holding  $z$  constant, so the two variants differ in how far ahead they look and not in how conservative they are at a given distance. Note that  $\hat{\mu}_i$ ,  $\hat{\sigma}_i$  are cold-start estimates that are least reliable early in a horizon.

*Look-Ahead (LA)*. A simulation-based rule, which was introduced in [7] alongside the SANS meta-heuristic, and that assumes no distributional form at all. It is the most complex and expensive of the three per decision, since it simulates forward instead of evaluating a closed form. Rather than projecting analytically, it rolls the bin's accumulation forward day by day to find the last day on which the bin could still be collected without overflowing, and makes it mandatory on that day. This defers collection to the latest safe moment so that the collected load is as large as possible. It has no threshold variant, which is why it contributes fewer configurations than LM or SL to the experimental grid.

#### 4.2 Route Constructors

Every constructor below solves the same single-period problem: visit all mandatory bins, optionally visit any subset of the remaining bins, respect vehicle capacity, and maximize collected waste net of travel cost. The adaptation is uniform across the families. Classical CVRP methods assume every node must be visited; each constructor therefore needs a mechanism to reject bins whose collected value would not repay the detour.

## Exact Methods

*Smart Waste Collection: Two-Commodity Flow (SWC-TCF)*. A direct implementation of the waste-collection formulation of Ramos et al. [18], written as a single monolithic mixed-integer program and handed to a general-purpose solver. Its core device is to track two quantities on every arc simultaneously: waste collected so far and residual capacity. Their sum equals vehicle capacity on every used arc. This constraint family enforces capacity and eliminates sub-tours at once, where a conventional formulation needs two separate families.

Unlike the other constructors, SWC-TCF needs no VRPP retrofit: the source formulation already treats “is bin  $i$  visited” as a first-class binary, with the objective trading collection revenue against travel cost and a per-vehicle usage penalty directly. Bins flagged mandatory, or already critically full, are forced into the solution regardless.

Its main limitation is scale. The model grows roughly quadratically in the number of candidate bins and there is no decomposition to keep the working problem small, making it the constructor most vulnerable to instance-size blow-up. The largest scenario exhibits this vulnerability (Sect. 5.4).

*Branch-Price-and-Cut (BPC)*. A decomposition-based exact method introduced in the seminal work [3] by Barnhart et al., where - for our VRPP formulation - the master problem is a route-selection linear program in which each column is one complete candidate route; routes are generated on demand by a pricing subproblem rather than enumerated, since the number of possible routes is astronomically large for any non-trivial instance. Pricing solves a resource-constrained shortest-path problem whose arc costs incorporate the master problem’s current duals, so a route is only generated if it could improve the incumbent. Because an exact elementary path search is intractable at this scale, the pricing works with near-elementary paths through the ng-route linear programming relaxation method [2], which relaxes the Set Partitioning Problem (SPP) formulation by allowing sub-tours outside of the ng-neighborhoods associated with each node, and it generates the initial columns (while the Restricted Master Problem - RMP - is unfeasible) by using Farkas pricing [13], in order to guarantee the eventual feasibility of the RMP.

Three mechanisms together give BPC its profitable-subset behavior. Coverage constraints are *at most once* for optional bins and *exactly once* for mandatory ones; the pricing subproblem maximizes profit rather than minimizing distance, so route extension stops naturally once a detour costs more than the waste it collects; and branching resolves the visit/no-visit decision for a bin *before* it branches on arcs, settling the question a classical always-cover formulation never has to ask. Beyond the base formulation, lifted cover inequalities [5] on saturated arcs are separated, following the reference methodology, together with VRPP-specific minimum-cut, triangle-clique and node-profit inequality families.

Both exact methods run under a per-run time budget in these experiments. Neither is run to certified optimality on the larger instances, and both have a documented non-exact fallback on timeout: BPC returns a greedily constructed

plan, SWC-TCF its best incumbent. Their results should be read as “best solution found within budget”, not as proven optima.

### Meta-Heuristics

*Adaptive Large Neighborhood Search (ALNS)*. A trajectory method, introduced by [19], that maintains one incumbent and repeatedly destroys and repairs part of it. Its adaptivity is in operator choice: it holds a portfolio of destroy operators (random removal, worst removal, and Shaw removal, which removes a set of mutually related bins so that repair has the freedom to place them jointly) and repair operators (greedy insertion and regret- $k$  insertion, optionally with randomized noise on insertion costs), and re-weights them by recent success. Operator  $i$  is drawn by roulette wheel from weights updated every  $\varphi$  iterations:

$$w_{i,t+1} = \lambda w_{i,t} + (1 - \lambda) \frac{\pi_i}{\theta_i}, \quad (6)$$

where  $\lambda \in [0, 1]$  is a decay factor,  $\pi_i$  the score accumulated by operator  $i$  during the segment, and  $\theta_i$  its usage count. Candidates are accepted by a simulated-annealing rule, so the trajectory can escape local optima. The profitable-subset decision falls out of repair: a removed bin for which no insertion position yields positive marginal profit is simply left out.

*Hybrid Genetic Search (HGS)*. A memetic algorithm following Vidal [22], and the population counterpart to ALNS. Its central design choice is representational: individuals are encoded as a single giant tour over all bins with no route boundaries, and are decoded into actual routes only when evaluated. Decoding is a dynamic program that finds the optimal split of the giant tour into capacity-feasible segments. For the VRPP, decoding can also skip any bin whose inclusion reduces net profit. Skipped bins remain in the genotype, so their relative position survives into later generations in case a subsequent mutation makes them worth collecting again.

Two sub-populations are maintained, one feasible and one infeasible-but-penalized, so that promising infeasible individuals survive long enough to be repaired. Each generation selects two parents by a tournament biased toward both fitness and diversity, recombines them, and educates the offspring by local search over relocate, 2-opt and the algorithm’s signature SWAP\* move. Fitness combines the profit function  $\mathcal{P}(\mathcal{A}_d, \mathbf{w}_d)$  with a diversity contribution  $D(\mathcal{A}_d)$ :

$$F(\mathcal{A}_d, \mathbf{w}_d) = \mathcal{P}(\mathcal{A}_d, \mathbf{w}_d) + \left(1 - \frac{|P_{feas}|}{|P|}\right) \cdot D(\mathcal{A}_d). \quad (7)$$

*Simulated Annealing Neighborhood Search (SANS)*. An implementation of the hybrid published by Jorge et al. [7] specifically for smart waste collection. It couples the standard annealing acceptance rule, which always accepts improvements and accepts a worsening move of magnitude  $\Delta$  with probability  $\exp(-\Delta/T)$  under a

cooling temperature  $T$ . It combines this rule with a purpose-built move library: single-bin relocation and swapping within and between routes, batched relocation and swapping of both random and contiguous bin groups, segment reversal, and route-segment crossover. Moves are sampled uniformly rather than adaptively, which is the substantive difference from ALNS: SANS steers entirely through acceptance, never down-weighting an unproductive operator.

The profitable-subset decision is folded into the neighborhood search itself. Bins can be moved into a removed pool and reconsidered for reinsertion later, so inclusion is one more move type rather than a separate phase. A refinement stage follows the annealing loop: an uncrossing pass, further annealing, deterministic local search applied in forward order and then with routes and positions reversed, followed by a final pass over the removed pool. Reversing the sweep can reach moves hidden by the first order.

*Pheromone-Guided Cooperative Large Neighborhood Search (PG-CLNS)*. An original design - inspired by the Hybrid Volleyball Premier League (HVPL) algorithm first introduced in [20] - which combines population-level route construction with per-individual refinement. Construction is ant-colony-based, with pheromone deposited on *edges* of the bin graph, so that road segments appearing in good solutions are more likely to be chosen again; only the global best solution deposits into the shared trail each round, keeping the signal focused, while a temporary within-round penalty discourages ants from reusing edges another ant has just taken. Refinement is large neighborhood search, run independently on every member of the population with its own adaptive destroy/repair weighting.

The two levels adapt on different objects and at different scopes: pheromone is edge-based and population-wide, operator weights are move-based and per individual. Each round constructs a solution population based on the current trail, refines every member, updates the trail from the round's best, and regenerates the weakest fraction of the population under the updated field. Profitability is screened at insertion in both phases, as in ALNS.

Because every member is refined every round, total runtime scales with population size times refinement depth times rounds; on non-trivial instances the algorithm is typically stopped by its time budget rather than by convergence.

*Particle Swarm Optimization Memetic Algorithm (PSOMA)*. Particle swarm optimization (PSO) is a continuous method, and applying it to a sequencing problem requires an encoding. To address this challenge, the authors of [14] developed a variation of the PSO method, named PSO Memetic Algorithm (PSOMA), as it combines both the evolutionary PSO algorithm with local search, with a novel encoding scheme which uses ranked-order values: each particle holds a real vector with one coordinate per bin, and the visiting order is simply the rank order of those coordinates. The numbers themselves are meaningless; only their relative order is read, so the standard continuous velocity update combines inertia with attraction to the particle's own best and the swarm's best. Sorting the updated values yields the discrete tour. The resulting giant tour is split into

routes by the same profit-aware dynamic program HGS uses (see above), which is likewise where PSOMA decides which bins to skip.

The memetic component refines the swarm’s incumbent once per iteration by a simulated-annealing local search over swap, relocate and segment-reversal moves, adaptively favoring whichever move type has recently been productive. A secondary heuristic biases the local search’s cost estimates so that pairs of profitable bins appear closer together, steering cheap moves toward retaining valuable bins without paying for a full route decode.

## Hyper-Heuristics

*Ant Colony Optimization Hyper-Heuristic (ACO-HH)*. The one true hyper-heuristic in the benchmark, and worth contrasting directly with PG-CLNS, which uses the same metaphor one level lower. Where classical ACO lays pheromone on edges of the *bin* graph to decide which bin to visit next, ACO-HH, as first defined by [4], lays pheromone on transitions in a graph whose nodes are the available low-level heuristics, and so decides which *move to apply* next. It searches the space of operator sequences and delegates all solution modification to the operator it selects.

An ant walks this operator graph, applying each move it steps onto to the working solution. The next operator is drawn with probability (TODO: change the variable symbols/names - they are correct according to the original paper, but they clash with some of the symbols/names, like  $\tau$  for threshold, we already use)

$$p_{sh} = \frac{[\tau_{sh}]^\alpha [\eta_{sh}]^\beta}{\sum_{k \in \mathcal{H}} [\tau_{sk}]^\alpha [\eta_{sk}]^\beta}, \quad (8)$$

where  $\tau_{sh}$  is the pheromone on the transition from state  $s$  to heuristic  $h$ ,  $\eta_{sh}$  is heuristic information about  $h$ ’s likely efficacy, and  $\alpha, \beta$  weight the two. If a completed walk improved the solution, pheromone is deposited along every transition it used, so operator *sequences* that work well together are reinforced rather than merely individually good operators; a faster-adapting efficiency signal that accounts for each operator’s runtime modulates selection on a shorter timescale, and all pheromone evaporates so that stale sequences fade.

Two separate portfolios are involved: construction heuristics (greedy, nearest- and farthest-insertion, regret-based, each with a profit-aware variant) used once to build the initial solution, and the modification operators the ant colony actually searches over. Inclusion decisions are handled by the profit-aware variants throughout, supplemented by a strategic-oscillation mechanism that periodically relaxes the capacity penalty when the search stagnates, letting it revisit inclusion decisions it would otherwise be trapped away from.

### 4.3 Route Improvers

Both improvers take a constructed route plan and refine it, and neither is designed to change which bins were selected.

*Classical Local Search (CLS).* Variable-neighborhood descent over a fixed toolbox ordered from cheapest to most expensive to evaluate: relocate a bin, swap two bins, reverse a segment (2-opt), relocate a chain of two or three consecutive bins (Or-opt), exchange route tails between vehicles, exchange bins between routes with re-optimized placement, and finally full three- and four-bin reordering. Whenever any move succeeds the search restarts from the cheapest neighborhood rather than continuing, and it terminates at a local optimum with respect to every move in the toolbox.

One property matters for interpreting the results: every CLS move is scored purely by its effect on travel distance. Capacity is enforced as a feasibility check, but collected revenue never enters the accept/reject decision, so CLS optimizes distance and not profit. It can, however, move a bin between vehicles.

*Fast-TSP.* A narrower operation: holding each route’s bin assignment fixed, find the best visiting order within it. Routes of up to roughly twenty stops are solved to optimality by dynamic programming; longer ones fall back to a randomized local search within a small fixed budget. Fast-TSP cannot move a bin between vehicles, drop a bin, or account for revenue. It solves each route’s ordering globally, whereas CLS improves the plan incrementally. Section 5.3 compares configurations using the two improvers.

#### 4.4 Simulation Protocol

Each simulated day executes a fixed cycle: bins accumulate waste; the selection strategy reads sensed fill levels and flags mandatory bins; the constructor builds a route and the improver refines it; the route is executed and the collected waste recorded; the day is logged and the state carried forward. Three properties of this loop bear on how the results should be read.

*Comparisons are paired at the level of the demand realization.* The simulator re-seeds its waste generation per policy-and-day combination rather than letting each run consume randomness in its own order, so every algorithm in a given scenario faces the literal identical sequence of daily fill levels, bin by bin and day by day. Differences between algorithms in the same scenario are therefore differences in decision-making, not in the draws they happened to face. This also underwrites the integrity check of Section 5.4. Within a scenario the total waste available to collect is fixed, so collected tonnage is directly comparable across the different policies.

*Overflow is a flag, loss is a quantity.* A bin’s true level is tracked continuously; waste beyond capacity is recorded as lost and the displayed level capped. An overflowing bin that stays unvisited keeps losing each subsequent increment until collection. Overflow counts and kilograms lost are therefore related but distinct measures of service failure.

*Every scenario uses a single vehicle and a single depot.* This is a deliberate scope restriction for the series of simulations presented here. While multi-vehicle and multi-depot dispatch is supported by the simulator and implemented algo-

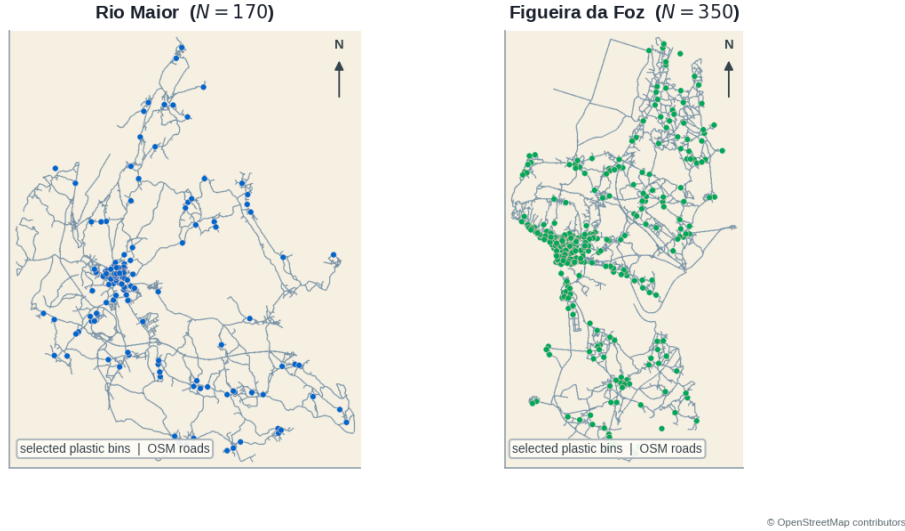
rithms, we opted here for a simplified problem due to the total number of policies being compared.

## 5 Experimental Evaluation

Experiments use two real Portuguese municipal networks with observed bin locations and road-distance matrices:

- **Rio Maior**, at  $N = 100$  and  $N = 170$  plastic-waste bins, with road distances from the Google Maps distance matrix;
- **Figueira da Foz**, at  $N = 350$  plastic-waste bins, with road distances from OpenStreetMap.

The two networks are shown side by side in Fig. 2.



**Fig. 2.** Geographic distribution of the selected plastic-waste bins in the two benchmark networks, shown over the Google Maps (for Rio Maior) and the OpenStreetMap (for Figueira da Foz) drive-network geometry. Each panel uses its own extent; bin positions are actual coordinates, while the remote depots are omitted to preserve detail within the service areas.

### 5.1 Scenarios and Data

Two waste-generation processes drive each network, chosen to contrast a parametric model against real observations:

- **Gamma-3**, drawing daily accumulation from fitted Gamma distributions ( $\alpha \in \{1, 3\}$ ,  $\beta \in \{8, 6\}$ ; means 8, 6, 24 and 18 kg/day with variances 64, 36, 192 and 108) with different presets assigned to different groups of bins, so accumulation behavior varies across the network and not merely in scale;
- **Empirical**, resampling directly from historical daily-fill records collected by the WSmart+ Bin Analysis module between 2021-01-12 and 2023-10-23. This bootstrap makes no parametric assumption and replays observed patterns.

Neither process reveals its parameters to any selection strategy; all accumulation statistics a strategy uses are estimated online from the days it has observed (Sect. 4.4). Two planning horizons are used, 30 and 90 days.

Both share a feature that shapes every result reported later: the depot is remote from the bins it serves. Its median road distance to a bin is roughly five times the median distance between two bins (52.9 against 8.6 units at Rio Maior, 46.6 against 10.6 at Figueira da Foz), so every route pays a long outbound and return leg before it collects anything. Selection strategies that collect on fewer days therefore amortize that fixed cost over a larger load, which is the mechanism behind the efficiency ordering in Sect. 5.3.

## 5.2 Design of Experiments

The 30-day design is a complete factorial crossing of every policy stage with every scenario:

$$\underbrace{8}_{\text{constructors}} \times \underbrace{5}_{\text{selection variants}} \times \underbrace{2}_{\text{improvers}} \times \underbrace{2}_{\text{distributions}} \times \underbrace{3}_{\text{network sizes}}$$

yielding 480 simulation runs, balanced so that each constructor appears in every scenario an equal number of times. Because the demand realization is fixed per scenario (Section. 4.4), the marginal mean of any one stage is a genuine like-for-like comparison rather than a comparison confounded by which scenarios that stage happened to be run in.

The 90-day design is *not* a factorial replication of this, and the distinction is important enough to state before any 90-day number is read. Only policy configurations that lay on the 30-day Pareto front were carried forward, producing 174 runs. Each constructor’s 90-day rows are therefore the scenarios in which it had already performed well at 30 days, and a cross-constructor comparison at 90 days would compare each constructor against a different, self-favoring subset. We report no such comparison. The horizon analysis in Section. 5.3 instead pairs each configuration against *itself* across the two horizons.

We report five measures. *Efficiency* (kg/km) is collected waste per kilometer driven and is the primary operational objective. *Overflow events* counts bin-days on which a bin was at capacity, and *kilograms lost* the waste discarded as a result; the two are related but distinct, since a single bin left uncollected keeps losing waste every subsequent day. *Distance* and *runtime* measure operational and computational cost. Means are reported alongside medians because a single extreme scenario can move a mean substantially at this sample size (Section. 5.4).

### 5.3 Results

**Route Constructors** Table 1 reports per-constructor performance, and Fig. 3 places the constructors in the efficiency/service plane.

BPC attains the highest mean efficiency (6.65 kg/km) on the shortest mean routes (4,018 km), although its time limit prevents an optimality certificate at these sizes. ALNS achieves the fewest mean overflows (5.9) and PG-CLNS is the strongest all-round compromise, second on efficiency and second on overflows. ACO-HH ranks third on efficiency, within 5% of BPC, and is the fastest constructor at 589 s mean runtime, roughly a quarter of the HGS and PG-CLNS means.

Seven of the eight constructors have a median overflow count of 4.0. The spread in their means, from 5.9 for ALNS to 19.9 for HGS, is therefore driven largely by tail events. Constructor differences on this measure describe the severity of unusual cases more than typical performance.

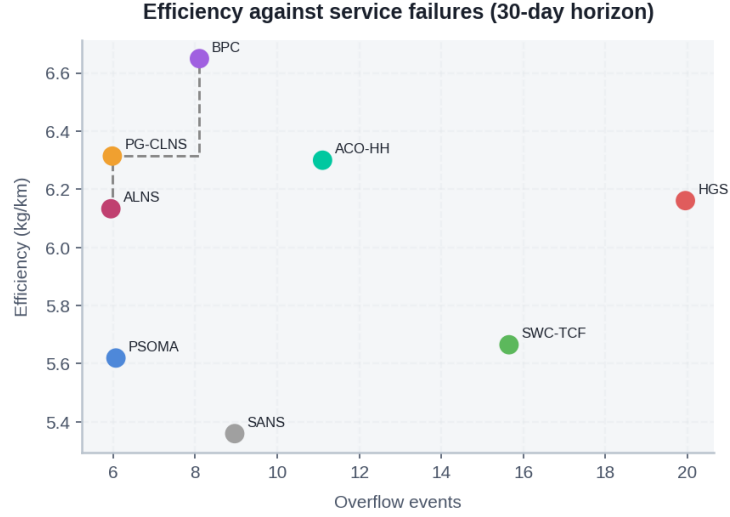
Pareto-front membership across the six scenario–distribution combinations, recomputed after the integrity filter, gives PG-CLNS 5 of 6, PSOMA and HGS 3 each, BPC 2, and ACO-HH and ALNS 1 each. PG-CLNS is thus the most consistently non-dominated constructor even though it never leads a single column outright, which is the characteristic signature of a good compromise method.

Runtime separates the constructors more cleanly than quality does (Fig. 4). At  $N = 350$  the spread is 4.5-fold, from ACO-HH at 1,219 s to SANS at 5,451 s. BPC averages 2,430 s; PG-CLNS, HGS and SANS all require more wall-clock time per run at this scale.

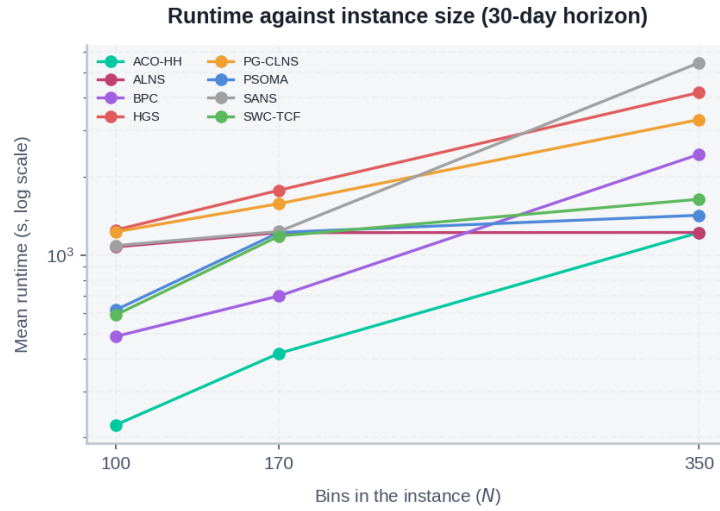
**Table 1.** Route constructor performance at the 30-day horizon over the balanced factorial grid (456 runs). Best value per column in bold.

| Route constructor | $n$ | kg/km       | med.        | Ovf.       | med.       | km           | Time (s)   |
|-------------------|-----|-------------|-------------|------------|------------|--------------|------------|
| BPC               | 57  | <b>6.65</b> | <b>6.31</b> | 8.1        | <b>4.0</b> | <b>4,018</b> | 1,140      |
| PG-CLNS           | 57  | 6.31        | 6.22        | 6.0        | <b>4.0</b> | 4,340        | 1,967      |
| ACO-HH            | 57  | 6.30        | 6.04        | 11.1       | <b>4.0</b> | 4,266        | <b>589</b> |
| HGS               | 57  | 6.16        | 5.99        | 19.9       | 6.0        | 4,553        | 2,309      |
| ALNS              | 57  | 6.13        | 6.22        | <b>5.9</b> | <b>4.0</b> | 4,497        | 1,168      |
| SWC-TCF           | 57  | 5.66        | 5.05        | 15.6       | <b>4.0</b> | 5,035        | 1,109      |
| PSOMA             | 57  | 5.62        | 5.14        | 6.1        | <b>4.0</b> | 5,294        | 1,067      |
| SANS              | 57  | 5.36        | 5.25        | 9.0        | <b>4.0</b> | 5,211        | 2,439      |

Means and medians over the scenarios that survive the integrity filter. Every constructor faced the identical demand realisation within each scenario and contributes the same 57 runs: the scenario cells containing the degenerate runs of Table 6 were dropped for all constructors, not only for the one that failed them, so no constructor is averaged over a subset that flatters it. Medians are reported alongside means because a single extreme scenario can move a mean substantially at this sample size.



**Fig. 3.** Mean efficiency against mean overflow events per constructor at the 30-day horizon, with the non-dominated front. Read alongside the medians in Table 1: the horizontal spread is largely tail behavior.



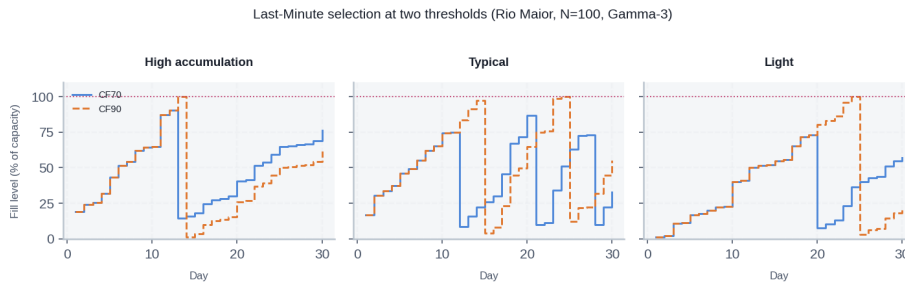
**Fig. 4.** Mean runtime against network size. The exact BPC solver is mid-pack; three meta-heuristics cost more per run than it does.

**Mandatory Selection Strategies** The selection stage produces a substantially larger observed spread than the constructor stage (Table 2, Fig. 6). The range in mean efficiency across the five tested selection variants is nearly four times the corresponding range across constructors. A benchmark that fixed selection and varied only construction would miss the larger operational trade-off in this design. The comparison describes the tested levels and does not constitute a general variance decomposition.

The five variants order monotonically on every reported measure: efficiency and overflow events fall together, while distance and runtime rise. Across these three different selection mechanisms, the observed ordering behaves like a single operational dial: deferring collection hauls more per kilometer on shorter routes but permits more overflows. No tested variant dominates another, so deployment requires an explicit valuation of service failure against travel.

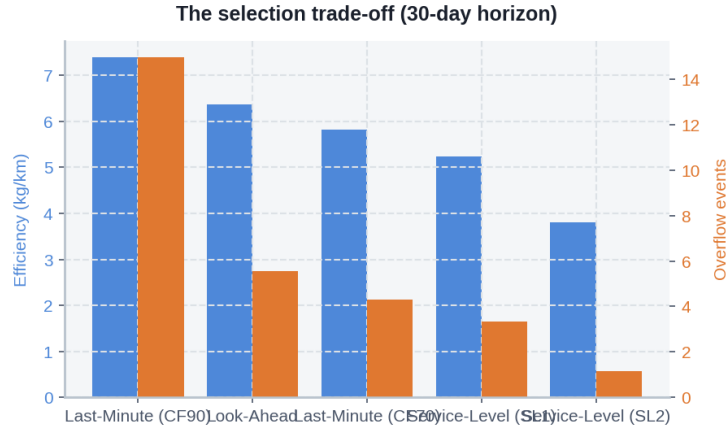
At similar service levels, the mechanisms carry different distance costs. Look-Ahead attains 5.5 mean overflows at 3,396 km, while Service-Level (SL1) needs 4,147 km to reach 3.3. The marginal cost of those last two overflow events is roughly 750 km, an order of magnitude greater than the 456 km that buys the nine-event reduction between LM-CF90 and Look-Ahead. These observed increments indicate sharply diminishing service returns.

The mechanism behind the dial is visible at the level of a single bin (Fig. 5). Because the demand realization is fixed, the same bin faces the identical waste increments under every strategy; what changes is when it is collected. Under the CF70 threshold the bin is emptied early and often, so its level never approaches capacity. Under CF90 collection is deferred until the bin is nearly full, hauling more waste per visit but leaving the bin one bad day from overflow. The highest-accumulation bins do overflow. The aggregate trade-off of Fig. 6 is this dynamic summed over every bin and day.



**Fig. 5.** Fill level over the 30-day horizon for three representative bins under the CF70 and CF90 thresholds, re-simulated from the recovered daily increments (Rio Maior,  $N = 100$ , Gamma=3). CF90 defers collection until the bin is nearly full, hauling more per visit but overflowing where CF70 does not.

One caveat governs this figure specifically. As described below in Sect. 5.4, there were degenerate runs in Look-Ahead and Service-Level (SL2) cells, so removing them removed the study’s hardest scenario from those two variants and not from the other three. Comparing the raw marginals would have credited Look-Ahead with a service level it did not achieve on equal terms. The table is therefore restricted to the slices in which all five variants ran, 80 runs each.



**Fig. 6.** Efficiency and overflow events by selection strategy, ordered by efficiency, over the slices all five variants share. Both decline monotonically along the observed efficiency–service trade-off.

**Route Improvers** The two improvers can be matched by configuration: each of the 224 pairs in Table 3 has the same constructor, strategy, network and demand realization, with a different configured improver. This controls the scenario mix, but it is not a controlled treatment comparison. Collected tonnage or bin count differs in many pairs, showing that stochastic upstream construction did not reproduce the same input route for both improvers.

In aggregate, configurations using CLS have higher efficiency than their matched-demand Fast-TSP counterparts (Table 3). CLS has a broader neighborhood that can move bins between routes, whereas Fast-TSP holds each route’s bin assignment fixed and changes only its internal order, but the observed delta cannot be attributed to that difference alone.

The losses are concentrated. Twenty-one of the 22 CLS losses occur at  $N = 350$ ; the remaining loss is HGS at  $N = 170$ . Every loss belongs to PSOMA, SWC-TCF, or HGS, and the negative differences are fewer but deeper than the positive ones (Fig. 7). This concentration motivates a controlled rerun at  $N = 350$  before selecting an improver for those constructors. The different neighborhoods provide

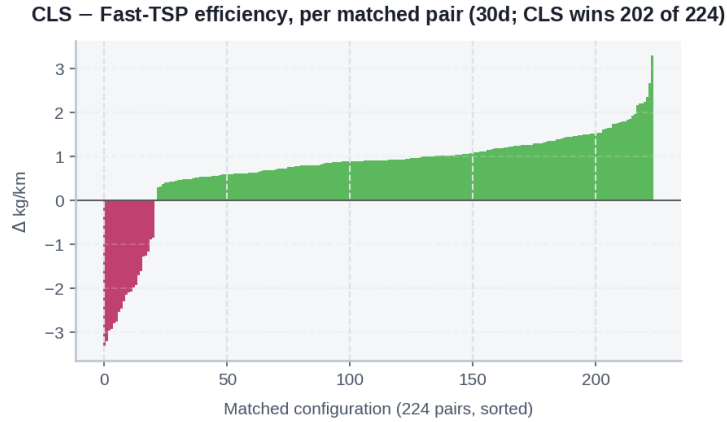
**Table 2.** Mandatory-selection variants at the 30-day horizon, ordered by observed collection efficiency, over the slices in which all five variants were run. Efficiency, overflow risk, distance and runtime all order monotonically along the observed efficiency–service trade-off.

| Selection strategy  | $n$ | kg/km       | med.        | Ovf.       | med.       | km           | Time (s)   |
|---------------------|-----|-------------|-------------|------------|------------|--------------|------------|
| Last-Minute (CF90)  | 80  | <b>7.38</b> | <b>7.40</b> | 14.9       | 12.0       | <b>2,940</b> | <b>892</b> |
| Look-Ahead          | 80  | 6.37        | 6.43        | 5.5        | 5.0        | 3,396        | 914        |
| Last-Minute (CF70)  | 80  | 5.82        | 5.97        | 4.3        | 4.0        | 3,788        | 1,171      |
| Service-Level (SL1) | 80  | 5.23        | 5.17        | 3.3        | 3.0        | 4,147        | 1,376      |
| Service-Level (SL2) | 80  | 3.81        | 4.08        | <b>1.2</b> | <b>1.0</b> | 5,711        | 1,941      |

CF70/CF90 are Last-Minute critical-fill thresholds of 70% and 90%; SL1/SL2 are Service-Level projection horizons of one and two days. Restricted to the slices shared by all five variants so that the marginal means are comparable: the excluded runs of Table 6 removed the study’s hardest scenario cell from Look-Ahead and Service-Level (SL2) but not from the other three variants, which would otherwise have flattered them. Medians are reported alongside means.

a plausible mechanism, but the summary logs contain no controlled route-length intervention with which to identify it.

Overflow counts tie on 181 of the 224 pairs. The remaining differences are further evidence that the stored experiment did not hold every upstream outcome fixed across improvers.



**Fig. 7.** Per-pair efficiency difference, CLS minus Fast-TSP, sorted. The losses are fewer but deeper than the wins, and all but one occur at  $N = 350$ .

**Table 3.** Classical Local Search against Fast-TSP at the 30-day horizon over 224 matched-demand configuration pairs.

| Metric                  | CLS   | Fast-TSP | $\Delta$ | CLS wins  | Ties |
|-------------------------|-------|----------|----------|-----------|------|
| Efficiency (kg/km)      | 6.39  | 5.65     | 0.74     | 202 / 224 | 0    |
| Distance travelled (km) | 4,358 | 4,662    | -304     | 202 / 224 | 0    |
| Overflow events         | 10.2  | 10.4     | -0.2     | 20 / 43   | 181  |
| Runtime (s)             | 1,412 | 1,451    | -38      | 185 / 224 | 0    |

Each pair has the same constructor, strategy, region and demand realisation, with a different configured improver. This is not a controlled improver treatment: upstream constructor outcomes also differ in some pairs, so the deltas are descriptive associations rather than causal effects. Win counts exclude ties, which are reported separately.

**Effect of the Planning Horizon** Table 4 compares the 165 configurations observed at both horizons against themselves. Efficiency is essentially horizon-invariant and slightly improving: the mean paired change is +0.26 kg/km, positive in 140 of 165 pairs, with HGS the only constructor whose efficiency falls (−0.47). For this selected subset, the result is consistent with efficiency being governed more by route geometry than by horizon length; it is not evidence for that claim over the unobserved policies.

Overflow counts grow, and the magnitude is informative: among pairs with a nonzero 30-day count, the median 90-day to 30-day ratio lies between 2.4 and 3.3 for six of the seven constructors, against a horizon ratio of exactly 3. The observed overflow counts therefore grow approximately in proportion to the horizon for these six constructors. SWC-TCF is the exception at 5.6, consistent with its scaling exposure elsewhere in these results.

We need to add two remarks. First, this comparison is paired precisely because a cross-constructor comparison at 90 days is not available. Second, pairing does not undo selection into the 90-day sample: every configuration was carried forward because of its 30-day Pareto performance on the scenario. The table therefore describes the selected configurations only. With one realization per configuration, we cannot estimate the selection effect or claim that the observed direction would hold over the full factorial design.

**Scenario Effects** The two generating processes produce markedly different regimes and should not be pooled (Table 5). On policy slices present under both processes, Gamma-3 produces both higher collection efficiency and more overflow events than the Empirical process. It is a heavier load: more waste is available per kilometer, but capacity is reached more often.

Network size does not act monotonically on efficiency because the two cities differ in spatial density as well as scale. Cost does increase across the three networks: both mean distance and runtime are highest at  $N = 350$ , as are overflow events. These marginals are restricted to policy slices shared by all levels of the factor being compared; without that balancing, the exclusions confined to Gamma-3 at  $N = 350$  would confound the scenario contrasts.

**Table 4.** Horizon sensitivity, computed on the 165 selected policy configurations observed at both horizons and compared against themselves.

| Route constructor | Pairs | kg/km 30d | kg/km 90d | Ovf. 30d | Ovf. 90d |
|-------------------|-------|-----------|-----------|----------|----------|
| ACO-HH            | 28    | 5.35      | 5.68      | 3.2      | 11.5     |
| BPC               | 57    | 6.65      | 6.96      | 8.1      | 28.5     |
| HGS               | 12    | 7.75      | 7.28      | 38.1     | 105.9    |
| PG-CLNS           | 40    | 6.26      | 6.56      | 5.0      | 16.3     |
| PSOMA             | 11    | 4.49      | 4.79      | 2.3      | 7.3      |
| SANS              | 12    | 4.66      | 4.93      | 2.8      | 8.7      |
| SWC-TCF           | 5     | 5.83      | 6.33      | 4.4      | 25.6     |

Only configurations on the 30-day Pareto front were re-run at 90 days, so the 90-day sample is conditioned on 30-day performance. Constructors are therefore compared against themselves across horizons and never against each other at 90 days. The paired values describe this selected subset and do not estimate a horizon effect for the full factorial design.

**Table 5.** Demand-process and network marginals at the 30-day horizon, each restricted to policy slices shared by every level of the factor being compared.

| Scenario factor                        | $n$ | kg/km | Ovf. | km    | Time (s) |
|----------------------------------------|-----|-------|------|-------|----------|
| Demand: Empirical                      | 216 | 4.71  | 6.1  | 3,662 | 1,313    |
| Demand: Gamma-3                        | 216 | 7.35  | 15.0 | 5,406 | 1,538    |
| Network: Rio Maior ( $N = 100$ )       | 136 | 5.74  | 5.2  | 2,223 | 815      |
| Network: Rio Maior ( $N = 170$ )       | 136 | 5.08  | 6.3  | 4,238 | 1,160    |
| Network: Figueira da Foz ( $N = 350$ ) | 136 | 6.99  | 21.6 | 7,618 | 2,608    |

The demand rows use the 216 policy slices present under both processes; the network rows use the 136 policy slices present on all three networks. Balancing prevents exclusions confined to Gamma-3 at  $N = 350$  from making the remaining levels look artificially better or worse.

#### 5.4 Data Integrity

Four runs were degenerate rather than merely poor, and all four were from SWC-TCF on Figueira da Foz under Gamma-3 (Table 6). Because every constructor in a scenario collects from an identical waste realization, collected tonnage is comparable by construction. These runs collected between 30% and 86% less than their scenario median and failed to complete. Their very large overflow counts include 23,886 events for the 90-day case, which reached day 13 of 90, and principally reflect truncation. This is the concrete realization of the scaling exposure noted for the monolithic MILP formulation: it is the largest network under the heavier of the two generating processes.

The detection rule was a tonnage shortfall against the scenario median. Outside these four runs the shortfall distribution has median 0.00 and does not exceed 0.071; the four sit at 0.30, 0.47, 0.47 and 0.86. The gap identifies them unambiguously. Collection-day count cannot diagnose truncation: several strong policies collect on only 15 of 30 days because they bundle collections well. Tonnage provides the relevant signal.

Excluding these runs alone would have introduced a subtler bias of the same family as the 90-day sampling problem: since all four are SWC-TCF’s, removing only them would average SWC-TCF over the scenarios where it did not fail while averaging its rivals over those scenarios too. We therefore dropped the affected scenario cells for *all* constructors, at a cost of 23 further runs, leaving 456 runs at 30 days with every constructor contributing exactly 57.

**Table 6.** Runs excluded from all aggregate statistics as degenerate rather than merely poor.

| Constructor | $H$ | Scenario | Policy             | kg     | Short. | Ovf.   |
|-------------|-----|----------|--------------------|--------|--------|--------|
| SWC-TCF     | 90  | FF 350   | Gamma-3/LA/FTSP    | 31,164 | 86%    | 23,886 |
| SWC-TCF     | 30  | FF 350   | Gamma-3/LA/FTSP    | 37,065 | 47%    | 2,166  |
| SWC-TCF     | 30  | FF 350   | Gamma-3/LA/CLS     | 37,065 | 47%    | 2,168  |
| SWC-TCF     | 30  | FF 350   | Gamma-3/SLSL2/FTSP | 51,086 | 30%    | 757    |

$H$  is the horizon in days and “FF 350” the Figueira da Foz network at  $N = 350$ . Within a scenario every constructor collects from the same waste realisation, so collected tonnage is comparable by construction. These runs fell more than 20% below their scenario median and did not complete; their overflow counts measure truncation, not policy behaviour.

The same reasoning applies to every other marginal comparison, in a form that is easy to miss. Dropping a scenario cell removes all constructors together, but it removes only *one* selection variant and *one* improver, since those identify the cell. Marginal tables over those stages are therefore additionally restricted to the slices in which every level of the compared stage actually ran. All tables and figures in this section were generated from the simulation output by an

automated tool that applied these rules centrally, so no reported number was transcribed by hand and the paper cannot drift from the data.

## 6 Conclusion and Future Work

WSmartRoute+ makes the temporal state of smart waste collection part of the benchmark rather than an implementation detail. Its three-stage policy model separates the decision to collect from the decisions that construct and refine a route, while common demand realizations make comparisons within a scenario directly attributable to policy behavior. This separation exposed the central operational result of the study: across the tested design, changing mandatory selection moves the efficiency–service trade-off more than changing the route constructor. Constructor choice still matters, particularly in runtime and in the tails of the overflow distribution. Improver configurations are associated with markedly different outcomes, but the stored runs do not isolate that stage from stochastic upstream construction.

The evaluation also shows why integrity rules belong in the experimental method. Whole-cell exclusion prevents failed SWC-TCF runs from flattering that constructor, and balancing every reported marginal prevents the same exclusions from flattering a strategy or scenario level instead. The 90-day results require an equally explicit boundary: because follow-up was conditioned on 30-day Pareto membership, they describe matched selected policies and cannot support a cross-constructor ranking or a population-level horizon effect.

The immediate next experiment is therefore a complete, replicated 90-day factorial design with multiple random seeds. That design would separate horizon effects from selection and regression to the mean. Further work should exercise the framework’s learned constructors against the classical baseline reported here, extend evaluation to multi-vehicle operations, and test whether adaptive selection rules can move the observed efficiency–service frontier rather than merely choose a point along it. For the monolithic two-commodity-flow model, explicit timeout and completion telemetry should also replace post-hoc failure detection at the largest scale.

*Selection acts on noisy observations.* Strategies see a sensed fill level perturbed by random error; overflow and loss accounting always uses the true level. A strategy can therefore make a wrong call purely from what its sensors reported, as in a real deployment.

**Acknowledgments.** This study was funded by the WSmart Route+ project (grant number BI|2024/549), and carried out at INESC-ID.

**Disclosure of Interests.** The authors have no competing interests to declare that are relevant to the content of this article.

## References

1. Archetti, C., Speranza, M.G., Vigo, D.: Chapter 10: Vehicle routing problems with profits. In: Toth, P., Vigo, D. (eds.) *Vehicle Routing: Problems, Methods, and*

- Applications, pp. 273–297. Society for Industrial and Applied Mathematics, 2 edn. (2014). <https://doi.org/10.1137/1.9781611973594.ch10>, <https://epubs.siam.org/doi/abs/10.1137/1.9781611973594.ch10>
2. Baldacci, R., Mingozzi, A., Roberti, R.: New route relaxation and pricing strategies for the vehicle routing problem. *Operations Research* **59**, 1269–1283 (09 2011). <https://doi.org/10.1287/opre.1110.0975>
  3. Barnhart, C., Johnson, E., Nemhauser, G., Savelsbergh, M., Vance, P.: Branch-and-price: Column generation for solving huge integer programs. *Operations Research* **46**, 316–329 (02 1970). <https://doi.org/10.1287/opre.46.3.316>
  4. Chen, P.C., Kendall, G., Vanden Berghe, G.: An ant based hyper-heuristic for the travelling tournament problem. pp. 19–26 (05 2007). <https://doi.org/10.1109/SCIS.2007.367665>
  5. Gu, Z., Nemhauser, G.L., Savelsbergh, M.W.P.: Lifted cover inequalities for 0-1 integer programs: Complexity. *INFORMS J. Comput.* **11**, 117–123 (1999), <https://api.semanticscholar.org/CorpusID:14531772>
  6. Hess, C., Dragomir, A.G., Doerner, K.F., Vigo, D.: Waste collection routing: a survey on problems and methods. *Central European Journal of Operations Research* **32**(2), 399–434 (2024). <https://doi.org/10.1007/s10100-023-00892-y>
  7. Jorge, D., Antunes, A.P., Ramos, T.R.P., Barbosa-Póvoa, A.P.: A hybrid meta-heuristic for smart waste collection problems with workload concerns. *Computers & Operations Research* **137**, 105518 (2022). <https://doi.org/10.1016/j.cor.2021.105518>
  8. Joshi, C.K., Cappart, Q., Rousseau, L.M., Laurent, T.: Learning the travelling salesperson problem requires rethinking generalization. *Constraints* **27**, 70–98 (2022). <https://doi.org/10.1007/s10601-022-09327-y>
  9. Joshi, C.K., Laurent, T., Bresson, X.: An efficient graph convolutional network technique for the travelling salesman problem. *arXiv preprint arXiv:1906.01227* (2019)
  10. Karalias, N., Loukas, A.: Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems* **33**, 6659–6672 (2020)
  11. Kool, W., van Hoof, H., Welling, M.: Attention, learn to solve routing problems! In: *International Conference on Learning Representations* (2019), <https://openreview.net/forum?id=ByxBFsRqYm>
  12. Kwon, Y.D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S.: Pomo: Policy optimization with multiple optima for reinforcement learning. *Advances in Neural Information Processing Systems* **33**, 21188–21198 (2020)
  13. Lin, K.M., Ehrgott, M., Raith, A.: Integrating column generation in a method to compute a discrete representation of the non-dominated set of multi-objective linear programmes. *4OR* **15**(4), 331–357 (2017). <https://doi.org/10.1007/s10288-016-0336-9>, <https://doi.org/10.1007/s10288-016-0336-9>
  14. Liu, B., Wang, L., Jin, Y.h., Huang, D.: An effective PSO-based memetic algorithm for TSP, vol. 345, pp. 1151–1156 (10 2006). [https://doi.org/10.1007/978-3-540-37258-5\\_149](https://doi.org/10.1007/978-3-540-37258-5_149)
  15. Lopes, M., Ramos, T.R.: Efficient sensor placement and online scheduling of bin collection. *Computers & Operations Research* **151**, 106113 (2023). <https://doi.org/10.1016/j.cor.2022.106113>
  16. Ma, Y., Cao, Z., Chee, Y.M.: Learning to search feasible and infeasible regions of routing problems with flexible neural k-opt. *Advances in Neural Information Processing Systems* **36** (2023), [https://proceedings.neurips.cc/paper\\_files/paper/2023/hash/9bae70d354793a95fa18751888cea07d-Abstract-Conference.html](https://proceedings.neurips.cc/paper_files/paper/2023/hash/9bae70d354793a95fa18751888cea07d-Abstract-Conference.html)

17. de Moraes, C.S., Ramos, T.R.P., Lopes, M., Barbosa-Póvoa, A.P.: A data-driven optimization approach to plan smart waste collection operations. *International Transactions in Operational Research* **31**(4), 2178–2208 (2024). <https://doi.org/10.1111/itor.13235>
18. Ramos, T.R.P., de Moraes, C.S., Barbosa-Póvoa, A.P.: The smart waste collection routing problem: Alternative operational management approaches. *Expert Systems with Applications* **103**, 146–158 (2018). <https://doi.org/10.1016/j.eswa.2018.03.001>, <https://www.sciencedirect.com/science/article/pii/S0957417418301404>
19. Røpke, S., Pisinger, D.: An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science* **40**, 455–472 (2006). <https://doi.org/10.1287/trsc.1050.0135>
20. Sun, S., Ma, L., Liu, Y., Shang, C.: Volleyball premier league algorithm with aco and alns for simultaneous pickup–delivery location routing problem. *Applied Soft Computing* **149**, 111004 (2023). <https://doi.org/https://doi.org/10.1016/j.asoc.2023.111004>, <https://www.sciencedirect.com/science/article/pii/S1568494623010220>
21. Sun, Z., Yang, Y.: Difusco: Graph-based diffusion solvers for combinatorial optimization. *Advances in Neural Information Processing Systems* **36**, 3706–3731 (2023)
22. Vidal, T.: Hybrid genetic search for the cvrp: Open-source implementation and swap\* neighborhood. *Computers & Operations Research* **140**, 105643 (2022). <https://doi.org/10.1016/j.cor.2021.105643>, <https://www.sciencedirect.com/science/article/pii/S030505482100349X>
23. Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. *Advances in neural information processing systems* **28** (2015)
24. Wu, X., Wang, D., Wen, L., Xiao, Y., Wu, C., Wu, Y., Yu, C., Maskell, D.L., Zhou, Y.: Neural combinatorial optimization algorithms for solving vehicle routing problems: A comprehensive survey with perspectives. *arXiv preprint arXiv:2406.00415* (2024)
25. Zhang, Z., Che, O., Cheang, B., Lim, A., Qin, H.: A memetic algorithm for the multiperiod vehicle routing problem with profit. *Eur. J. Oper. Res.* **229**(3), 573–584 (2013). <https://doi.org/10.1016/J.EJOR.2012.11.059>, <https://doi.org/10.1016/j.ejor.2012.11.059>